Replay Court Specification v0.4

> The Conformance Target. Supersedes v0.1, v0.2, and v0.3 entirely. All protocol ambiguities resolved. This document is sufficient to write unit tests, define ABI schemas, and enforce cross-language parity.

---

0. Constitutional Rule

The Replay Court must be built before autonomous execution.

Every later system depends on this invariant:

> No state transition is real unless the Replay Court can independently verify it.

That is the bedrock.

---

1. Cryptographic Primitives (Locked)

Primitive	Choice	Rationale
Hash function	`keccak256`	EVM-native, Solidity-compatible
Signature scheme	`secp256k1` recoverable signatures	Aligns with `ecrecover`, Foundry, cross-chain settlement
ABI encoding	Strict Ethereum ABI	Deterministic, no padding variation

Ed25519 is not used anywhere in this system.

1.1 Identity Binding Law

All identity fields (`governor_id`, `proposer_id`, `node_id`, `attester_id`) MUST resolve to the left-padded 32-byte representation of the recovered Ethereum address.

```
id = bytes32(uint256(uint160(ecrecover(digest, v, r, s))))
```

This maintains direct EVM compatibility. `ecrecover` is the canonical verification primitive. Identity is derived from the key, not asserted by the caller.

1.2 Signature Sites

Field	Signs	Using
`governor_signature`	`transition_id`	secp256k1 / `ecrecover`
`proposer_signature`	`proposal_id`	secp256k1 / `ecrecover`
`node_signature`	`node_id`	secp256k1 / `ecrecover`
`attester_signature`	`attestation_id`	secp256k1 / `ecrecover`

Signatures are never included in the content they authenticate.

1.3 Time Law

The Replay Court and the WASM Mutation Engine have no access to system time.

```
No Date.now()
No std::time::SystemTime
No clock imports in WASM
```

`created_at` and `expires_at` are static input data, locked at the time of original execution. Historical replay evaluates against the exact timestamps in the stored payload. Machine time is never consulted.

1.4 Empty List Hash Rule

Empty `authority_chain` or `attestations` arrays produce inconsistent ABI output across Rust, TypeScript, Python, and Solidity. To enforce absolute cross-language determinism, empty lists bypass ABI encoding entirely.

```
// When list is empty:
hash_authority_chain([])    = keccak256(abi.encode(uint256(0)))
hash_attestations([])       = keccak256(abi.encode(uint256(0)))
```

When the list is non-empty, use the standard ordered encoding defined in §5.

---

2. AgentGenomeStateV2 (Highest Priority)

This is the root state object. It is strictly flat. Dynamic fields are banned from the root to guarantee ABI parity across all implementations.

```
pub struct AgentGenomeStateV2 {
    pub schema_version: u16,
    pub agent_id: [u8; 32],
    pub genome_id: [u8; 32],
    pub lineage_root: [u8; 32],
    pub lifecycle_flags: u32,
    pub viability_flags: u32,
    pub capability_flags: u32,
    pub mutation_root: [u8; 32],
    pub runtime_flags: u64,
    pub sequence: u64,
    pub payload_hash: [u8; 32],
}
```

No `Vec`, no nested structs, no optional fields at the root level.

State Hash Law

```
state_hash = keccak256(abi.encode(
    schema_version,    // u16
    agent_id,          // bytes32
    genome_id,         // bytes32
    lineage_root,      // bytes32
    lifecycle_flags,   // u32
    viability_flags,   // u32
    capability_flags,  // u32
    mutation_root,     // bytes32
    runtime_flags,     // u64
    sequence,          // u64
    payload_hash       // bytes32
))
```

Field order is declaration order. No exceptions.

Lifecycle Flags

Bit	Name	Meaning
`0x00000001`	`ACTIVE`	Agent is live

0x00000002	QUARANTINED	Agent is suspended pending review
0x00000004	TOMBSTONED	Agent is permanently terminated
0x00000008	FORKED	Agent has been forked; this record is the source lineage

Viability Flags

Reserved for mutation engine use. The Replay Court treats this field as an opaque `u32`. Mutation engines may define internal bit semantics.

---

3. StateTransitionV1

```
pub struct StateTransitionV1 {
    pub schema_version: u16,
    pub domain_separator: [u8; 32],
    pub transition_id: [u8; 32],
    pub pre_state_hash: [u8; 32],
    pub post_state_hash: [u8; 32],
    pub mutation_hash: [u8; 32],
    pub authority_context_hash: [u8; 32],
    pub verdict_hash: [u8; 32],
    pub mutation_engine_hash: [u8; 32],
    pub semantic_version: u32,
    pub codec_version: u32,
    pub governor_id: [u8; 32],
    pub governor_signature: Vec<u8>,
    pub epoch: u64,
    pub sequence: u64,
    pub created_at: u64,
}
```

Canonical transition hash:

```
transition_id = keccak256(abi.encode(
    domain_separator,       // bytes32
    schema_version,         // u16
    pre_state_hash,         // bytes32
    post_state_hash,        // bytes32
    mutation_hash,          // bytes32
    authority_context_hash, // bytes32
    verdict_hash,           // bytes32
```

```
    mutation_engine_hash,   // bytes32
    semantic_version,       // u32
    codec_version,          // u32
    governor_id,            // bytes32
    epoch,                  // u64
    sequence,               // u64
    created_at              // u64
))
```

`governor_signature` is excluded from `transition_id`. The Governor signs `transition_id` using secp256k1. Verification uses `ecrecover` and the Identity Binding Law (§1.1).

---

4. MutationProposalV1

```
pub struct MutationProposalV1 {
    pub schema_version: u16,
    pub proposal_id: [u8; 32],
    pub proposer_id: [u8; 32],
    pub target_agent_id: [u8; 32],
    pub mutation_type: u32,
    pub payload_hash: [u8; 32],
    pub semantic_version_required: u32,
    pub codec_version_required: u32,
    pub nonce: u64,
    pub created_at: u64,
    pub expires_at: u64,
    pub proposer_signature: Vec<u8>,
}
```

`payload` is not a field. Raw payload bytes live in content-addressed storage. The Court receives them as a separate argument and asserts:

```
keccak256(payload_bytes) == proposal.payload_hash
```

Canonical proposal hash:

```
proposal_id = keccak256(abi.encode(
    schema_version,            // u16
    proposer_id,               // bytes32
    target_agent_id,           // bytes32
    mutation_type,             // u32
    payload_hash,              // bytes32
```

```
    semantic_version_required, // u32
    codec_version_required,    // u32
    nonce,                     // u64
    created_at,                // u64
    expires_at                 // u64
))
```

`proposer_signature` signs `proposal_id` using secp256k1. Excluded from `proposal_id`.

4.1 Mutation Types

Code	Name	Description
`0x0001`	`GENOME_PATCH`	Structured patch to one or more genome fields
`0x0002`	`GENOME_REPLACE`	Full genome replacement (requires elevated authority)
`0x0003`	`CAPABILITY_GRANT`	Adds a capability flag
`0x0004`	`CAPABILITY_REVOKE`	Removes a capability flag
`0x0005`	`ATTESTATION_ATTACH`	Attaches a signed attestation to the agent record
`0x0006`	`LINEAGE_FORK`	Forks agent into a new lineage (immutable after)
`0x00FF`	`TOMBSTONE`	Terminates agent; no further mutations permitted

Unknown mutation type codes → `CourtOutcome::InvalidMutationType` (8).

Do not conflate with version errors.

4.2 Version Checks

- Unsupported `semantic_version_required` → `CourtOutcome::InvalidSemanticVersion` (6)
- Unsupported `codec_version_required` → `CourtOutcome::InvalidCodecVersion` (7)

These three cases (4.1, 4.2) are distinct. They must never produce the same outcome code.

4.3 Expiry

A proposal is expired when `expires_at < transition.created_at`.

Per the Time Law (§1.3): expiry is evaluated against `transition.created_at` locked in the stored payload, never against machine time.

- On live execution: expired proposal → `CourtOutcome::InvalidAuthority` (4)
- On historical replay: the Court does not re-reject. It sets `AuthorityReview.expired_attestation_flags` and continues.

---

5. AuthorityContextV1

```
pub struct AuthorityContextV1 {
    pub schema_version: u16,
    pub context_id: [u8; 32],
    pub governor_id: [u8; 32],
    pub authority_chain: Vec<AuthorityNode>,
    pub policy_hash: [u8; 32],
    pub policy: Vec<u8>,
    pub attestations: Vec<AttestationV1>,
    pub epoch: u64,
    pub created_at: u64,
}

pub struct AuthorityNode {
    pub node_id: [u8; 32],
    pub role: u32,
    pub delegation_depth: u8,
    pub node_signature: Vec<u8>,
}

pub struct AttestationV1 {
    pub attestation_id: [u8; 32],
    pub attester_id: [u8; 32],
    pub subject_id: [u8; 32],
    pub claim_hash: [u8; 32],
    pub valid_from: u64,
    pub valid_until: u64,
    pub attester_signature: Vec<u8>,
}
```

Canonical context hash:

```
context_id = keccak256(abi.encode(
    schema_version,              // u16
    governor_id,                 // bytes32
    hash_authority_chain(chain), // bytes32
    policy_hash,                 // bytes32
```

```
    hash_attestations(atts),     // bytes32
    epoch,                       // u64
    created_at                   // u64
))
```

Sub-hash functions:

```
// Non-empty chain:
hash_authority_chain(chain) = keccak256(abi.encode(
    for each node in order: (node_id, role, delegation_depth)
    // node_signature excluded
))

// Empty chain:
hash_authority_chain([]) = keccak256(abi.encode(uint256(0)))

// Non-empty attestations:
hash_attestations(atts) = keccak256(abi.encode(
    for each attestation in order:
      (attestation_id, attester_id, subject_id, claim_hash, valid_from, valid_unt
    // attester_signature excluded
))

// Empty attestations:
hash_attestations([]) = keccak256(abi.encode(uint256(0)))
```

Signatures are always excluded from content hashes.

```
policy_hash = keccak256(policy_bytes)
```

The Court does not interpret `policy`. It passes `policy_bytes` to the WASM mutation engine as part of the context argument. Policy validity is the mutation engine's responsibility entirely.

5.1 Authority Roles

Code	Role	Required Delegation Depth
0x01	ROOT_GOVERNOR	0
0x02	DOMAIN_GOVERNOR	1
0x03	AGENT_SUPERVISOR	2
0x04	PROPOSER	3

5.2 Delegation Depth Law

Delegation depth must **increase monotonically by exactly 1** per hop, starting at 0 for the root governor.

```
Valid:   ROOT(0) → DOMAIN_GOVERNOR(1) → PROPOSER(2) ✓
Invalid: ROOT(0) → PROPOSER(2)                       ✗ skip
Invalid: ROOT(0) → DOMAIN_GOVERNOR(1) → node(1)      ✗ duplicate
Invalid: ROOT(0) → DOMAIN_GOVERNOR(2) → PROPOSER(1)  ✗ reversal
```

Any skip, duplicate, or reversal → `CourtOutcome::InvalidAuthority` (4), with `VerdictReasonCode::InvalidDelegationDepth` (2).

The root governor (depth 0) is self-signing only. `ecrecover(node_id, node_signature)` must recover to the same `node_id` at depth 0.

Note: v0.3 stated depth must "decrease monotonically." That was incorrect. Depth increases from root outward. v0.4 is the authoritative definition.

---

6. VerdictV1

```
pub struct VerdictV1 {
    pub schema_version: u16,
    pub outcome: u32,
    pub reason_code: u32,
    pub flags: u32,
    pub authority_context_hash: [u8; 32],
    pub mutation_hash: [u8; 32],
    pub created_at: u64,
}
```

Canonical verdict hash:

```
verdict_hash = keccak256(abi.encode(
    schema_version,         // u16
    outcome,                // u32
    reason_code,            // u32
    flags,                  // u32
    authority_context_hash, // bytes32
    mutation_hash,          // bytes32
    created_at              // u64
))
```

6.1 Verdict Outcome Codes

Code	Name
0	Allow
1	Deny
2	DenyExpired
3	DenyInsufficientAuthority
4	DenyPolicyViolation

6.2 Verdict Reason Code Registry

```
pub enum VerdictReasonCode {
    None                    = 0,
    ExpiredProposal         = 1,
    InvalidDelegationDepth  = 2,
    MissingRootGovernor     = 3,
    UnauthorizedMutationType = 4,
    PolicyViolation         = 5,
    InsufficientRole        = 6,
    QuarantinedAgent        = 7,
    TombstonedAgent         = 8,
}
```

Overloading semantic errors is banned. Each rejection path emits exactly one `VerdictReasonCode`. The code is machine-readable and must be deterministic across all implementations.

---

7. CourtOutcome (Complete Enum)

```
pub enum CourtOutcome {
    TransitionVerified     = 0,
    TransitionChallenged   = 1,
    InvalidSignature       = 2,
    InvalidPostState       = 3,
    InvalidAuthority       = 4,
    InvalidMutationEngine  = 5,
    InvalidSemanticVersion = 6,
```

```
    InvalidCodecVersion    = 7,
    InvalidMutationType    = 8,
    InvalidPreState        = 9,
    InvalidProposalHash    = 10,
    InvalidContextHash     = 11,
    ReplayUnavailable      = 12,
}
```

---

8. AuthorityReview

Produced alongside `CourtOutcome`. Records the temporal validity split without altering history.

```
pub struct AuthorityReview {
    pub valid_at_execution: bool,
    pub valid_under_current_law: bool,
    pub decayed_reputation_score: i32,
    pub expired_attestation_flags: u32,
    pub review_required: bool,
}
```

A transition may be `TransitionVerified` (valid when executed) while `valid_under_current_law = false` (authority since revoked). These are two distinct facts. The Court records both without modifying either.

---

9. Mutation Engine WASM Interface

The Court loads `mutation_engine.wasm` by exact content hash:

```
mutation_engine_hash = keccak256(wasm_bytes)
```

Any byte-level mismatch → `CourtOutcome::InvalidMutationEngine` (5).

Required Export

```
apply_mutation(
    pre_state_ptr: u32,
    pre_state_len: u32,
    proposal_ptr: u32,
```

```
    proposal_len: u32,
    context_ptr: u32,
    context_len: u32,
    out_ptr: u32
) -> u32
```

Output: canonical ABI bytes for `AgentGenomeStateV2`.

Forbidden Capabilities

The WASM module must not use:

- Filesystem
- Network
- Clock or timestamp functions (enforces the Time Law)
- Randomness
- Environment variables
- Floating point
- Threading
- Any host callback except `keccak256`

Permitted Host Import

```
keccak256(input_ptr: u32, input_len: u32, out_ptr: u32) -> u32
```

This is the only permitted import. Execution is pure and deterministic.

---

10. Hash Function Reference

```
// State object
state_hash(s: AgentGenomeStateV2) -> [u8; 32]
  = keccak256(abi.encode(all fields, declaration order))

// Proposal (alias to pre-computed id)
hash_mutation(p: MutationProposalV1) -> [u8; 32]
  = p.proposal_id

// Context (alias to pre-computed id)
```

```
hash_authority_context(c: AuthorityContextV1) -> [u8; 32]
  = c.context_id

// Verdict (full field set)
hash_verdict(v: VerdictV1) -> [u8; 32]
  = keccak256(abi.encode(
      v.schema_version,
      v.outcome,
      v.reason_code,
      v.flags,
      v.authority_context_hash,
      v.mutation_hash,
      v.created_at
    ))

// Payload (loaded separately)
hash_payload(bytes: &[u8]) -> [u8; 32]
  = keccak256(bytes)

// Authority chain sub-hash (see §5, Empty List Rule)
hash_authority_chain(chain: Vec<AuthorityNode>) -> [u8; 32]
  = if chain.is_empty() { keccak256(abi.encode(uint256(0))) }
    else { keccak256(abi.encode(for each: (node_id, role, delegation_depth))) }

// Attestations sub-hash (see §5, Empty List Rule)
hash_attestations(atts: Vec<AttestationV1>) -> [u8; 32]
  = if atts.is_empty() { keccak256(abi.encode(uint256(0))) }
    else { keccak256(abi.encode(for each: (attestation_id, attester_id, subject_i
```

`canonical_abi_encode` : strict Ethereum ABI, field order is struct declaration order, no padding variation, no optional omission.

---

11. The Seven Execution Invariants (The Constitution)

No state transition is recognized as valid unless all seven hold sequentially:

```
1. proposal_id         == keccak256(abi.encode(proposal_fields_without_signature)
2. context_id          == keccak256(abi.encode(context_fields_without_signatures)
3. transition_id       == keccak256(abi.encode(transition_fields_without_signatur
4. payload_hash        == keccak256(raw_payload_bytes)
5. mutation_engine_hash == keccak256(wasm_bytes)
6. post_state_hash     == state_hash(wasm_apply_mutation(pre_state, payload, cont
7. ecrecover(transition_id, governor_signature) == governor_address
   (resolved per the Identity Binding Law: bytes32(uint256(uint160(address))))
```

If any invariant fails, the transition is not verified. There is no partial verification.

---

12. Verification Algorithm

```
fn verify_transition(
    transition: StateTransitionV1,
    pre_state: AgentGenomeStateV2,
    proposal: MutationProposalV1,
    payload_bytes: &[u8],
    authority_context: AuthorityContextV1,
    mutation_engine_wasm: &[u8],
) -> CourtOutcome {

    // Invariant 5
    if keccak256(mutation_engine_wasm) != transition.mutation_engine_hash {
        return CourtOutcome::InvalidMutationEngine;
    }

    // Invariant 4
    if keccak256(payload_bytes) != proposal.payload_hash {
        return CourtOutcome::InvalidProposalHash;
    }

    // Invariant 1: proposal self-consistency
    if hash_proposal_fields(proposal) != proposal.proposal_id {
        return CourtOutcome::InvalidProposalHash;
    }

    // Invariant 2: context self-consistency
    if hash_context_fields(authority_context) != authority_context.context_id {
        return CourtOutcome::InvalidContextHash;
    }

    // Version and type checks (fail fast, before any state work)
    if !supported_semantic_version(proposal.semantic_version_required) {
        return CourtOutcome::InvalidSemanticVersion;
    }
    if !supported_codec_version(proposal.codec_version_required) {
        return CourtOutcome::InvalidCodecVersion;
    }
    if !known_mutation_type(proposal.mutation_type) {
        return CourtOutcome::InvalidMutationType;
    }
```

```
// Pre-state hash
if state_hash(pre_state) != transition.pre_state_hash {
    return CourtOutcome::InvalidPreState;
}

// Cross-reference: mutation hash
if proposal.proposal_id != transition.mutation_hash {
    return CourtOutcome::InvalidProposalHash;
}

// Cross-reference: context hash
if authority_context.context_id != transition.authority_context_hash {
    return CourtOutcome::InvalidContextHash;
}

// Authority verdict
let verdict = check_authority(pre_state, proposal, authority_context);
if hash_verdict(verdict) != transition.verdict_hash {
    return CourtOutcome::InvalidAuthority;
}
if verdict.outcome != VerdictOutcome::Allow {
    return CourtOutcome::TransitionChallenged;
}

// Invariant 6: WASM execution
let post_state_bytes = wasm_apply_mutation(
    mutation_engine_wasm,
    pre_state,
    proposal,
    payload_bytes,
    authority_context,
);
if state_hash_from_bytes(post_state_bytes) != transition.post_state_hash {
    return CourtOutcome::InvalidPostState;
}

// Invariant 3: transition self-consistency
if hash_transition_fields(transition) != transition.transition_id {
    return CourtOutcome::InvalidSignature;
}

// Invariant 7: governor signature via ecrecover
let recovered = ecrecover(transition.transition_id, transition.governor_signa
let recovered_id = bytes32(uint256(uint160(recovered)));
if recovered_id != transition.governor_id {
    return CourtOutcome::InvalidSignature;
```

```
    }

    CourtOutcome::TransitionVerified
}
```

---

13. Challenge Protocol

```
observer submits challenge
  → bond locked
  → Replay Court re-executes with stored inputs
  → CourtOutcome != TransitionVerified:
      challenger wins bond; transition flagged
  → CourtOutcome == TransitionVerified:
      challenger loses bond; transition confirmed
```

No human judgment at this layer. Only deterministic re-execution against stored inputs, evaluated per the Time Law using locked timestamps.

---

14. Authority Decay

The Court produces `AuthorityReview` alongside `CourtOutcome` on every call.

Authority may become invalid over time without invalidating historical transitions. The two facts are recorded separately and never conflated.

---

15. Normative Test Vector

The canonical conformance test. All implementations must produce identical results.

15.1 Happy Path

Inputs:

```
domain_separator:      0x0000000000000000000000000000000000000000000000000000000000000000
epoch:                 1
transition.sequence:   1
transition.created_at: 1700000100

pre_state (AgentGenomeStateV2):
```

```
  schema_version:  1
  agent_id:        0xaaaa0000000000000000000000000000000000000000000000000000000000
  genome_id:       0x0000000000000000000000000000000000000000000000000000000000000000
  lineage_root:    0x0000000000000000000000000000000000000000000000000000000000000000
  lifecycle_flags: 0x00000001  (ACTIVE)
  viability_flags: 0x00000000
  capability_flags:0x00000000
  mutation_root:   0x0000000000000000000000000000000000000000000000000000000000000000
  runtime_flags:   0x0000000000000000
  sequence:        0
  payload_hash:    0x0000000000000000000000000000000000000000000000000000000000000000

proposal (MutationProposalV1):
  schema_version:            1
  proposer_id:               0xbbbb0000000000000000000000000000000000000000000000000000000000
  target_agent_id:           0xaaaa0000000000000000000000000000000000000000000000000000000000
  mutation_type:             0x00000003  (CAPABILITY_GRANT)
  payload_hash:              keccak256(abi.encode(uint32(0x00000001)))
  semantic_version_required: 1
  codec_version_required:    1
  nonce:                     1
  created_at:                1700000000
  expires_at:                1800000000

payload_bytes:
  abi.encode(uint32(0x00000001))  // grant capability flag bit 0

authority_context (AuthorityContextV1):
  schema_version: 1
  governor_id:    0xcccc0000000000000000000000000000000000000000000000000000000000
  authority_chain:
    [0] node_id:          0xcccc0000000000000000000000000000000000000000000000000000000000
        role:             0x01  (ROOT_GOVERNOR)
        delegation_depth: 0
    [1] node_id:          0xbbbb0000000000000000000000000000000000000000000000000000000000
        role:             0x04  (PROPOSER)
        delegation_depth: 1
  policy_hash:    keccak256(policy_bytes)
  policy:         abi.encode("allow CAPABILITY_GRANT flag=0x00000001 to any ACTIV
  attestations:   []
  epoch:          1
  created_at:     1700000000
```

Expected outputs:

```
payload_hash_verified:        true
```

```
proposal_id_verified:         true
context_id_verified:          true
delegation_depth_valid:       true  (0 → 1, monotonically increasing by 1)
hash_attestations result:     keccak256(abi.encode(uint256(0)))  (empty list rule

verdict:
  outcome:                    Allow (0)
  reason_code:                None (0)
  flags:                      0x00000000

post_state.capability_flags:  0x00000001
post_state.sequence:          1  (incremented by mutation engine)

court_outcome:                TransitionVerified (0)

authority_review:
  valid_at_execution:         true
  valid_under_current_law:    true
  decayed_reputation_score:   0
  expired_attestation_flags:  0
  review_required:            false
```

15.2 Failure Vectors (All Must Pass)

#	Perturbation	Expected `CourtOutcome`	Code
F1	Flip one byte of `mutation_engine_wasm`	`InvalidMutationEngine`	5
F2	`expires_at = created_at - 1`	`InvalidAuthority`	4
F3	`delegation_depth[1] = 0` (duplicate, non-monotonic)	`InvalidAuthority`	4
F4	`delegation_depth[1] = 3` (skip, non-monotonic)	`InvalidAuthority`	4
F5	`mutation_type = 0x00FF` (TOMBSTONE, policy disallows)	`TransitionChallenged`	1
F6	Corrupt `governor_signature`	`InvalidSignature`	2
F7	`post_state_hash` set to `0x0000...0000`	`InvalidPostState`	3
F8	`pre_state_hash` set to wrong value	`InvalidPreState`	9

F9	`mutation_type = 0xDEAD` (unknown)	`InvalidMutationType`	8
F10	`semantic_version_required = 999`	`InvalidSemanticVersion`	6
F11	`codec_version_required = 999`	`InvalidCodecVersion`	7
F12	`payload_bytes` tampered	`InvalidProposalHash`	10
F13	`context_id` set to `0x0000...0000`	`InvalidContextHash`	11
F14	`lifecycle_flags` set to `TOMBSTONED (0x00000004)` in pre_state	`TransitionChallenged`	1

F3 and F4 are now two separate vectors for the two non-monotonic failure modes (duplicate and skip). F14 exercises the `TombstonedAgent` verdict reason code path.

Each failure case must:

1. Produce exactly the listed `CourtOutcome` code
2. Produce no state change
3. Be reproducible across all conforming implementations

---

16. Implementation Order

Step	Deliverable	Gate
1	`AgentGenomeStateV2` ABI schema and `state_hash` (§2)	—
2	All hash functions (§10) including empty list rule	Step 1
3	secp256k1 key generation, signing, and `ecrecover` identity binding	Step 2
4	Happy-path test vector (§15.1) with concrete computed byte values	Step 3
5	All 14 failure vectors (§15.2) pass	Step 4
6	Reference WASM mutation engine (`mutation_engine_v1_test.wasm`)	Step 5
7	Challenge protocol	Step 6

Do not begin WASM development until Steps 1–5 pass. Hash and signature correctness is the foundation. WASM correctness is built on top.

17. Resolved Items

All items previously tracked as open in v0.3 are now resolved:

Item	Resolution
`AgentGenomeStateV2` full ABI schema	Defined in §2
Governor key scheme	secp256k1 + `ecrecover`, §1.1
Delegation depth direction	Increases from root outward, §5.2
Empty list hashing	`keccak256(abi.encode(uint256(0)))`, §1.4
Time source in WASM	No clock; static payload only, §1.3
`VerdictReasonCode` registry	Defined in §6.2
`VerdictV1` full field set in `verdict_hash`	All 7 fields, §6
Concrete byte values for §15.1	Pending Step 4 of implementation
Reference WASM binary	Pending Step 6 of implementation
Reputation decay scoring formula	Out of scope for Court layer; mutation engine concern
Bond sizing for challenge protocol	Out of scope for v0.4; economic layer TBD

---

v0.4 — The Conformance Target. Supersedes all prior versions. No protocol ambiguities remain. Sufficient to begin implementation.